

Web Application Development

Lecture 1: React Introduction

SWE 381 – Web Application Development

Dr. Ohoud Alharbi Dr. Achraf Gazdar

Department of Software Engineering
College of Computer and Information Sciences
King Saud University

Term 2 – 2025/2026

Learning Objectives

By the end of this lecture, you will be able to:

- Define React and explain its role in modern web development
- Distinguish between **imperative** and **declarative** programming
- Explain how the **Virtual DOM** improves UI performance
- Use ES6 **modules** to organise and reuse code
- Set up a React project with **Vite**
- Write a **functional component** and embed expressions in **JSX**
- Describe the **Single Page Application** (SPA) concept

Agenda

- 1 What is React?
- 2 Declarative vs Imperative
- 3 The Virtual DOM
- 4 ES6 Modules: import / export
- 5 Vite Project Setup
- 6 First Component + JSX
- 7 Single Page Applications (SPA)
- 8 Practical Examples & Summary

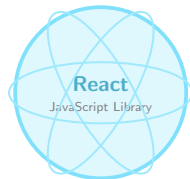
Outline

- 1 What is React?
- 2 Declarative vs Imperative
- 3 The Virtual DOM
- 4 ES6 Modules: import / export
- 5 Vite Project Setup
- 6 First Component + JSX
- 7 Single Page Applications (SPA)
- 8 Practical Examples & Summary

What is React? – W3Schools Definition

W3Schools – React Introduction

React, sometimes referred to as a front-end JavaScript framework, is a **JavaScript library** created by **Facebook** for building fast and interactive **user interfaces**.

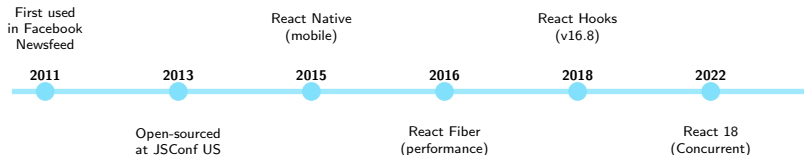


Open-source · Facebook · 2013

Key Facts (W3Schools):

- Created by **Jordan Walke**, Facebook Software Engineer
- First used in **2011** for Facebook's Newsfeed
- Open-sourced in **July 2013** (v0.3.0)
- Latest version: **React 19** (December 2024)
- Also known as *React.js* or *ReactJS*
- Responsible only for the **View layer** (the "V" in MVC)

React Timeline – A Brief History



Key milestones:

- **2011** – Jordan Walke creates React at Facebook
- **2013** – Public release, v0.3.0
- **2015** – React Native for mobile apps
- **2016** – React Fiber rewrite for better perf.

- **2018** – Hooks introduced (v16.8)
- **2022** – React 18 with Concurrent Mode
- **2024** – React 19 (latest stable)



Used by

Facebook, Instagram, Netflix, Airbnb, WhatsApp, Twitter

Why React? – Vanilla JS vs Components

Vanilla JavaScript approach:

- Manually select and update DOM elements
- State scattered across global variables
- No standard code reuse mechanism
- Large apps become **hard to maintain**
- Every change requires imperative DOM code

Problem with Vanilla JS at scale

As UI grows, manually tracking which DOM element to update when data changes becomes error-prone and slow.

React approach:

- UI split into small, **reusable components**
- Each component owns its own **state**
- React automatically updates the DOM when state changes
- Code is **declarative** and easier to reason about
- Same component can be used in multiple places

Solution

Components are the building blocks of every React application. They are independent, reusable pieces of UI.

Vanilla JS vs React – Side by Side

Vanilla JavaScript:

```
1 // HTML: <p id="msg">Hello</p>
2 //       <button id="btn">Click</button>
3
4 let count = 0;
5 const msg = document
6   .getElementById('msg');
7 const btn = document
8   .getElementById('btn');
9
10 btn.addEventListener('click', () => {
11   count++;
12   // Must manually update DOM
13   msg.textContent =
14     'Clicked ' + count + ' times';
15 });
```

→ You tell the browser **HOW** to update.

React:

```
1 import { useState } from 'react';
2
3 function Counter() {
4   const [count, setCount] = useState(0);
5
6   return (
7     <>
8       <p>Clicked {count} times</p>
9       <button onClick={
10         () => setCount(count + 1)
11       }>
12         Click
13       </button>
14     </>
15   );
16 }
```

→ You describe **WHAT** the UI should look like.

Outline

- 1 What is React?
- 2 Declarative vs Imperative
- 3 The Virtual DOM
- 4 ES6 Modules: import / export
- 5 Vite Project Setup
- 6 First Component + JSX
- 7 Single Page Applications (SPA)
- 8 Practical Examples & Summary

Declarative vs Imperative Programming

Imperative Programming

- You specify **step-by-step HOW** to do something
- You manage every detail of how the UI updates
- Like giving turn-by-turn driving directions
- Traditional DOM manipulation is imperative

"Create a <div>, set its class to 'active', append it to #container..."

Declarative Programming

- You specify **WHAT** the result should be
- React figures out HOW to update the DOM
- Like telling a GPS your destination, not the route
- React is **declarative by design**

"If isActive is true, show a <div className='active'>..."

Imperative (Vanilla JS)
"Do step 1, step 2, step 3..."

React shifts you here

Declarative (React)
"Here is what I want."

Declarative Code Example

Imperative – DOM steps:

```
1 // IMPERATIVE: step by step
2 function showWelcome(user) {
3   const el = document
4     .createElement('h2');
5   el.textContent = 'Welcome, ' + user;
6   el.style.color = 'green';
7   document
8     .getElementById('app')
9     .appendChild(el);
10 }
11 showWelcome('Ahmed');
12
13 // What if user changes?
14 // Must remove old el and add new one!
```

Declarative – React JSX:

```
1 // DECLARATIVE: describe the outcome
2 function Welcome({ user }) {
3   return (
4     <h2 style={{ color: 'green' }}>
5       Welcome, {user}
6     </h2>
7   );
8 }
9
10 // React handles DOM updates
11 // when 'user' changes automatically
```



React's contract

Update the **state** – React updates the **DOM**. You never touch the DOM directly.

Outline

- 1 What is React?
- 2 Declarative vs Imperative
- 3 The Virtual DOM**
- 4 ES6 Modules: import / export
- 5 Vite Project Setup
- 6 First Component + JSX
- 7 Single Page Applications (SPA)
- 8 Practical Examples & Summary

What is the Virtual DOM?

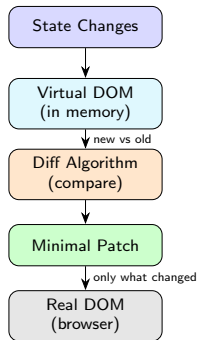
W3Schools Definition

React creates a **Virtual DOM** in memory. Instead of manipulating the browser's DOM directly, React creates a virtual DOM in memory, where it does all necessary manipulating before making changes in the browser DOM.

React only changes what needs to be changed!

Key idea:

- The Virtual DOM is a **lightweight JavaScript object** that mirrors the real DOM
- Lives in **memory only** – not in the browser
- Changes are made to the Virtual DOM first
- React computes the **minimal set of real DOM changes**
- This process is called **reconciliation**



Real DOM vs Virtual DOM

Feature	Real DOM	Virtual DOM
Updates	Slow – triggers full reflow	Fast – JS object in memory
Location	In the browser	In JavaScript memory
When updated	On every change	Only when state/props change
What is updated	The whole tree (often)	Only the diffed nodes
Side effects	Layout recalculation, repaint	None – just JS objects
Developer control	Manual (<code>document.getElementById</code>)	React handles it

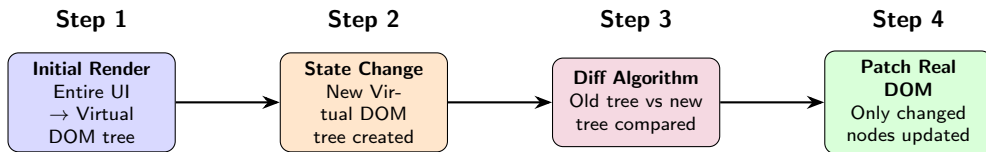
⚠ Real DOM is expensive

Every DOM manipulation triggers **layout recalculation** and **repaint**. Doing this many times per second causes slow, janky UIs.

💡 Virtual DOM is fast

Manipulating a JavaScript object (Virtual DOM) is orders of magnitude faster. React batches changes and applies the minimal real DOM update.

How Virtual DOM Reconciliation Works



Example: A list of 500 items. One item's text changes. React updates **only that one** `<li>` – not the whole list.

Outline

- 1 What is React?
- 2 Declarative vs Imperative
- 3 The Virtual DOM
- 4 ES6 Modules: import / export
- 5 Vite Project Setup
- 6 First Component + JSX
- 7 Single Page Applications (SPA)
- 8 Practical Examples & Summary

Why ES Modules?

W3Schools – ES Modules

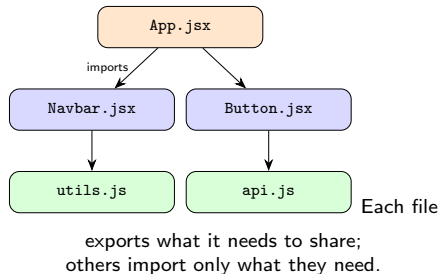
JavaScript modules allow you to break up your code into **separate files**. This makes it easier to maintain the codebase. ES Modules rely on the `import` and `export` statements.

Problems without modules:

- All code in one file – unmanageable
- Global variable name conflicts
- Cannot easily reuse code across files
- No clear ownership of functionality

With ES Modules:

- Each file is its own **isolated scope**
- Only exported names are visible to other files
- React components are naturally one-per-file



Named Exports and Default Exports

Named Exports – person.js:

```
1 // Export inline
2 export const name = "Jesse";
3 export const age = 40;
4
5 // Or export all at once at bottom
6 const name = "Jesse";
7 const age = 40;
8 export { name, age };
```

Importing Named Exports:

```
1 // Must use the SAME name
2 // Must use curly braces {}
3 import { name, age } from './person.js';
4
5 console.log(name); // Jesse
6 console.log(age); // 40
```

Default Export – message.js:

```
1 // Only ONE default export per file
2 const message = () => {
3   return "Hello from message.js!";
4 };
5
6 export default message;
```

Importing Default Export:

```
1 // No curly braces
2 // Can use ANY name you want
3 import myMsg from './message.js';
4
5 console.log(myMsg()); // Hello from...
```



React convention

Each component file uses **default export** for the component itself, and **named exports** for helpers or constants.

ES Modules in a React Project

Greeting.jsx – named + default:

```
1 // Named export: a constant
2 export const COURSE = "SWE 381";
3
4 // Named export: a helper function
5 export function formatName(name) {
6   return name.trim().toUpperCase();
7 }
8
9 // Default export: the component
10 function Greeting({ name }) {
11   return (
12     <h1>
13       Hello, {formatName(name)}!
14       Welcome to {COURSE}.
15     </h1>
16   );
17 }
18
19 export default Greeting;
```

App.jsx – importing both:

```
1 // Default import (no braces)
2 import Greeting from './Greeting';
3
4 // Named imports (with braces)
5 import { COURSE, formatName }
6   from './Greeting';
7
8 function App() {
9   return (
10     <div>
11       <Greeting name="ahmed" />
12       <p>Course: {COURSE}</p>
13       <p>{formatName(" sara ")}</p>
14     </div>
15   );
16 }
17
18 export default App;
```

Browser

Hello, AHMED!
Welcome to SWE 381.
Course: SWE 381
SARA

Outline

- 1 What is React?
- 2 Declarative vs Imperative
- 3 The Virtual DOM
- 4 ES6 Modules: import / export
- 5 Vite Project Setup**
- 6 First Component + JSX
- 7 Single Page Applications (SPA)
- 8 Practical Examples & Summary

What is Vite?

W3Schools – Get Started

When you have Node.js installed, you can start creating a React application by choosing a build tool. We will use the **Vite** build tool in this tutorial.

Why Vite over Create React App?

- **Instant server start** – uses native ES modules
- **Ultra-fast HMR** – Hot Module Replacement
- Much **faster builds** than Webpack-based CRA
- Modern tooling, actively maintained
- Recommended by the React team for new projects



Prerequisite

Install **Node.js** (v18 or later) from <https://nodejs.org> first.
Check with `node -v` and `npm -v`.

Create React App

Old, slow, deprecated

deprecated

Vite

Modern, fast, recommended

use this

Next.js

Full-stack framework

advanced

Creating a React App with Vite – Step by Step

Step 1: Create the project (W3Schools command):

```
npx create-vite my-react-app --template react
```

Step 2: Install dependencies:

```
cd my-react-app  
npm install
```

Step 3: Start the development server:

```
npm run dev  
# VITE v6.x ready in 217 ms  
# Local: http://localhost:5173/
```



Open your browser

Navigate to **http://localhost:5173** – you will see the default Vite + React starter page with a live counter button. Every file save instantly reflects in the browser (HMR).

Vite Project Structure

Folder structure:

```
my-react-app/  
  node_modules/    <- dependencies  
  public/  
    vite.svg  
  src/  
    assets/  
      react.svg  
    App.css  
    App.jsx        <- main component  
    index.css  
    main.jsx       <- entry point  
    index.html     <- HTML shell  
    package.json  
    vite.config.js
```

Key files explained:

- index.html – the single HTML page; has <div id="root">
- src/main.jsx – entry point; mounts React into #root
- src/App.jsx – the top-level component you edit
- vite.config.js – build configuration

main.jsx entry point

```
1 import { StrictMode } from 'react'  
2 import { createRoot } from 'react-dom/client'  
3 import App from './App.jsx'  
4  
5 createRoot(document.getElementById('root'))  
6   .render(  
7     <StrictMode>  
8       <App />  
9     </StrictMode>  
10  )
```

Outline

- 1 What is React?
- 2 Declarative vs Imperative
- 3 The Virtual DOM
- 4 ES6 Modules: import / export
- 5 Vite Project Setup
- 6 First Component + JSX**
- 7 Single Page Applications (SPA)
- 8 Practical Examples & Summary

What is JSX?

W3Schools – JSX

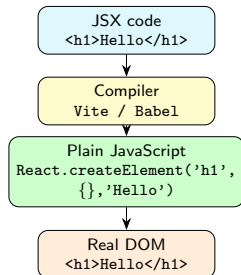
JSX stands for **JavaScript XML**. It allows us to write HTML directly within JavaScript. JSX is an extension of the JavaScript language based on ES6, and is translated into regular JavaScript at **runtime** by Babel / Vite.

JSX is not HTML – it compiles to:

- JSX is **syntactic sugar** over `React.createElement()`
- Vite / Babel compiles JSX to plain JavaScript before the browser runs it
- The browser never sees JSX – only compiled JS

You are not required to use JSX, but:

- JSX makes React code much more **readable**
- It is used in virtually every real React project



JSX Syntax Rules

Rule 1: One top-level element

```
1 // WRONG -- two top-level elements
2 return (
3   <h1>Title</h1>
4   <p>Paragraph</p>    // Error!
5 );
6
7 // CORRECT -- wrapped in one element
8 return (
9   <div>
10     <h1>Title</h1>
11     <p>Paragraph</p>
12   </div>
13 );
14
15 // CORRECT -- use Fragment <>...</>
16 return (
17   <>
18     <h1>Title</h1>
19     <p>Paragraph</p>
20   </>
21 );
```

Rule 2: Self-closing tags

```
1 // HTML allows: <br> 
2 // JSX requires: <br /> 
3
4 return (
5   <>
6     
7     <br />
8     <input type="text" />
9   </>
10 );
```

Rule 3: className not class

```
1 // HTML: <div class="card">
2 // JSX:
3 return (
4   <div className="card">
5     Content
6   </div>
7 );
8 // 'class' is a reserved word in JS
```

JSX Expressions – Embedding JavaScript

Embed JS expressions with { }:

```
function StudentCard() {  
  const name      = "Ahmed";  
  const gpa       = 3.8;  
  const year      = new Date().getFullYear();  
  const isHonor   = gpa >= 3.7;  
  
  return (  
    <div className="card">  
      /* Comment in JSX */  
      <h2>{name}</h2>  
      <p>GPA: {gpa}</p>  
      <p>Year: {year}</p>  
      <p>Status: {  
        isHonor ? "Honor Roll" : "Regular"  
      }</p>  
      <p>Score: {gpa * 25}%</p>  
    </div>  
  );  
}
```



Browser Output

Ahmed
GPA: 3.8
Year: 2025
Status: Honor Roll
Score: 95%

What goes inside { }:

- Variables: {name}
- Expressions: {gpa * 25}
- Ternary: {x ? a : b}
- Function calls: {getName()}
- **Not** statements (if, for loops)

Your First React Component

App.jsx – replace the default:

```
// A functional component is just a  
// JavaScript function that returns JSX  
  
function App() {  
  const course   = "SWE 381";  
  const lecturer = "Dr. Alharbi";  
  
  return (  
    <div>  
      <h1>Welcome to {course}!</h1>  
      <p>Lecturer: {lecturer}</p>  
      <p>  
        Today: {  
          new Date().toLocaleDateString()  
        }  
      </p>  
    </div>  
  );  
}  
  
// Every component file exports its  
// component as default  
export default App;
```

 http://localhost:5173

Welcome to SWE 381!

Lecturer: Dr. Alharbi

Today: Mon Mar 24 2025

Component Rules

- Function name **must start with capital letter**
- Must return **one top-level** JSX element
- Use **default export** to make it importable
- Used as a **custom HTML tag**: <App />

Composing Components

Header.jsx:

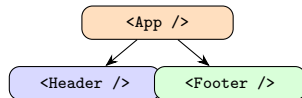
```
1 function Header() {  
2   return (  
3     <header style={{ background: '#20232a',  
4                          color: '#61dafb',  
5                          padding: '10px' }}>  
6       <h1>SWE 381 -- React App</h1>  
7     </header>  
8   );  
9 }  
10 export default Header;
```

Footer.jsx:

```
1 function Footer() {  
2   return (  
3     <footer>  
4       <p>KSU -- Dept. of Software Eng.</p>  
5     </footer>  
6   );  
7 }  
8 export default Footer;
```

App.jsx – compose them:

```
1 import Header from './Header';  
2 import Footer from './Footer';  
3  
4 function App() {  
5   return (  
6     <div>  
7       <Header />  
8       <main>  
9         <p>Main content here</p>  
10      </main>  
11      <Footer />  
12    </div>  
13  );  
14 }  
15  
16 export default App;
```



Outline

- 1 What is React?
- 2 Declarative vs Imperative
- 3 The Virtual DOM
- 4 ES6 Modules: import / export
- 5 Vite Project Setup
- 6 First Component + JSX
- 7 Single Page Applications (SPA)**
- 8 Practical Examples & Summary

What is a Single Page Application?

SPA Definition

A **Single Page Application** (SPA) is a web application that dynamically rewrites the current page rather than loading entire new pages from the server. The browser loads one HTML file and JavaScript handles **all navigation and content updates**.

Core characteristics:

- Only **one HTML page** is loaded from the server
- Page content updates happen **via JavaScript**
- **No full page reload** when navigating
- Feels like a **native desktop application**
- Initial load may be slower; navigation is **very fast**

Famous SPA examples:

Gmail

Facebook

Netflix

LinkedIn

Twitter / X

Airbnb

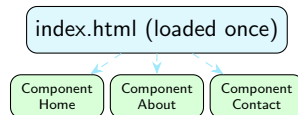
SPA vs Multi-Page Application (MPA)

Multi-Page Application (MPA)



Each navigation → new HTML fetched from server

Single Page Application (SPA)



JavaScript swaps components – no reload

	MPA	SPA
Page loads	Full reload every navigation	Once, then JS takes over
Speed	Slower navigation	Faster after initial load
Server	Renders HTML per page	Sends one shell + JS bundle
Routing	Server-side	Client-side (React Router)

SPA with React – How It Works

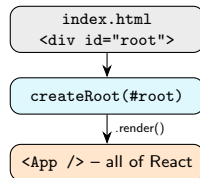
index.html – the single page:

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>My SPA</title>
5 </head>
6 <body>
7   <!-- React mounts here -->
8   <div id="root"></div>
9   <!-- All JS bundled by Vite -->
10  <script type="module"
11    src="/src/main.jsx">
12  </script>
13 </body>
14 </html>
```

This is the **only** HTML file. The #root div starts empty – React fills it.

main.jsx – React takes control:

```
1 import { createRoot } from 'react-dom/client'
2 import App from './App.jsx'
3
4 // React injects the entire app
5 // inside <div id="root">
6 createRoot(
7   document.getElementById('root')
8 ).render(<App />)
```



Outline

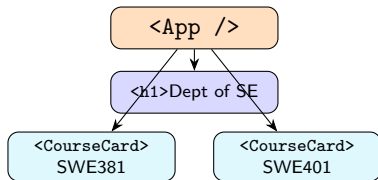
- 1 What is React?
- 2 Declarative vs Imperative
- 3 The Virtual DOM
- 4 ES6 Modules: import / export
- 5 Vite Project Setup
- 6 First Component + JSX
- 7 Single Page Applications (SPA)
- 8 Practical Examples & Summary**

Putting It All Together – Full Example

```
1 // src/components/CourseCard.jsx
2 export const DEPT = "Software Engineering"; // named export
3
4 function CourseCard({ code, title, credits }) { // functional component
5   const isCore = credits >= 3;
6
7   return (
8     <div className="card">
9       <h2>{code}: {title}</h2>           {/* JSX expressions */}
10      <p>Credits: {credits}</p>
11      <p>Type: {isCore ? "Core" : "Elective"}</p>  {/* ternary */}
12      <p>Dept: {DEPT}</p>
13    </div>
14  );
15 }
16 export default CourseCard;                // default export
17
18 // src/App.jsx
19 import CourseCard, { DEPT } from './components/CourseCard'; // import both
20
21 function App() {
22   return (
23     <div>
24       <h1>Dept. of {DEPT}</h1>
25       <CourseCard code="SWE381" title="Web App Dev" credits={3} />
26       <CourseCard code="SWE401" title="Software Design" credits={2} />
27     </div>
28   );
29 }
30 export default App;
```

Component Output Visualised

The component tree:



Browser Output

Dept. of Software Engineering

SWE381: Web App Dev

Credits: 3

Type: Core

Dept: Software Engineering

SWE401: Software Design

Credits: 2

Type: Elective

Dept: Software Engineering

Lecture 1 – Key Takeaways

- 1 **React** is a JavaScript library for building UIs, created by Facebook (2013), currently v19
- 2 React shifts development from **imperative** (how) to **declarative** (what)
- 3 The **Virtual DOM** is a JS object in memory; React diffs it with the real DOM and patches only what changed
- 4 **ES Modules** (import/export) organise code into reusable files; components use *default* exports
- 5 **Vite** is the recommended build tool for new React projects – instant start, fast HMR
- 6 A **functional component** is a JS function that returns JSX; names must start with a capital letter
- 7 **JSX** embeds HTML-like syntax in JS; use {expr} for dynamic values; must have one root element
- 8 A **SPA** loads one HTML page; React handles all routing and content updates in the browser

Practice Exercises

- 1 Install Node.js and Vite. Create a new React project with `npx create-vite swe381-app --template react`. Start the dev server and confirm you see the default page at `localhost:5173`.
- 2 Edit `src/App.jsx`: replace all default content with a component that displays your name, student ID, and the current year using JSX expressions.
- 3 Create a new file `src/components/CourseCard.jsx` with a `CourseCard` component that accepts `code`, `title`, and `credits` as variables. Import and use it twice in `App.jsx`.
- 4 Add a named export `UNIVERSITY` from `CourseCard.jsx` and import it in `App.jsx` to display it in a heading.
- 5 Practise JSX rules: try writing a component that returns two sibling elements without a wrapper – observe the error. Fix it using a Fragment `<>...</>`.
- 6 Draw the Virtual DOM update flow on paper: what happens when you change a paragraph's text? List the four steps (render, diff, patch, update).
- 7 **Challenge:** Create a simple SPA layout: a `Header` component, a `Main` component, and a `Footer` component. Compose them in `App.jsx` using three separate files with proper `import/export`.